組合語言 上機大抄

★ 通用模板（直接抄）

```
.386
.model flat, c
includelib kernel32.lib
includelib msvcrt.lib
includelib legacy_stdio_definitions.lib
includelib ucrt.lib
ExitProcess proto stdcall :dword
SetConsoleOutputCP proto stdcall :dword
printf proto c :vararg
scanf proto c :vararg
time proto c :vararg
srand proto c :vararg
rand proto c
system proto c :vararg

.data
arr dword 45,12,89,3,27,66,14,90,55,8

ansMsg  byte "sum = %d, max = %d, avg = %d",
10, 0

.code
main proc c
    invoke SetConsoleOutputCP, 65001

    mov ecx, lengthof arr
    mov esi, offset arr
    mov eax, [esi + ecx*4-4]

    invoke ExitProcess, 0
main ENDP

END main
```

0 開專案、錯誤

空白專案（英文路徑）→ 專案右鍵「建置相依性 → 建置自訂」勾 **masm** → 平台 **x86** → 加入 main.asm → main.asm 右鍵「屬性 → 一般 → **項目類型**」改成 **Microsoft Macro Assembler** → Ctrl+Shift+B → **Ctrl+F5**

0 錯誤卻沒 exe：項目類型沒改 · **A1000** 中文路徑 · **LNK2019** 少 lib · **A2006** 拼錯／沒宣告／少 PROTO · **A2008/A2151** 撞保留字（name size type nop outs）· **A2070** mem,mem、大小不合、div 2 直接接數字 · x64 改 x86

1 輸入輸出

讀一個整數 → n

```
invoke scanf, addr fmtD, addr n ; 給位址
```

一次讀兩個整數（空白隔開）

```
invoke scanf, addr fmt2, addr a, addr b
```

讀字串（不含空白）→ buf

```
invoke scanf, addr fmtS, addr buf
```

印整數

```
invoke printf, addr msg, n ; 給值
```

一次印多個值

```
invoke printf, addr msg2, a, b
```

印字串

```
; %s 給位址
invoke printf, addr fmtL, addr buf
invoke printf, addr hello
```

印暫存器的值

```
invoke printf, addr msg, eax
```

格式碼、換行

```
; %d 整數  %u 無號  %x 16進位
; %c 字元  %s 字串
; 字串後加 10（或 0Ah）= 換行
; 最後一定要 0 結尾
```

不用 INVOKE（push/call）

```
push n              ; 參數由右至左推
push offset msg
call printf
add esp, 8          ; C：參數個數×4
```

視窗停住 / 中文

```
invoke system, addr pz ; pz byte "pause",0
invoke SetConsoleOutputCP, 65001
```

2 運算

x = a（記憶體到記憶體）

```
mov eax, a          ; mem,mem 不行
mov x, eax
```

++ / -- / 清 0

```
inc n
dec n
xor eax, eax        ; eax = 0
```

加、減

```
mov eax, a
add eax, b          ; eax = a + b
sub eax, b          ; eax = eax - b
```

乘法

```
mov eax, a
imul eax, b         ; 有號 eax = eax×b
imul eax, b, 5      ; eax = b×5
mov eax, a
mul b               ; 無號 edx:eax = eax×b
```

除法、餘數

```
mov eax, a          ; 被除數
xor edx, edx        ; 有號改用 cdq
mov ecx, 3          ; 除數放暫存器
div ecx             ; 有號用 idiv
; 商 → eax，餘數 → edx
```

×2ⁿ、÷2ⁿ（不用乘除）

```
shl eax, 3          ; ×8
shr eax, 1          ; ÷2（無號）
add eax, eax        ; ×2
```

變負號、交換

```
neg eax             ; eax = -eax
xchg eax, ebx       ; 交換
```

3 判斷

if / else 骨架

```
mov eax, a
cmp eax, b
jg  ifBig           ; a > b 跳過去
  ; …else 的事…
  jmp ifEnd         ; 一定要跳過 if 區塊
ifBig:
  ; …if 的事…
ifEnd:
```

跳躍指令對照

```
; 有號  >  jg   >= jge   <  jl   <= jle
; 無號/字元  ja   jae   jb   jbe
; ==  je(jz)   !=  jne(jnz)
```

範圍判斷（'A'~'Z'）

```
mov al, 'Q'
cmp al, 'A'
jb  notIn           ; 字元用 jb/ja
cmp al, 'Z'
ja  notIn
  ; …在範圍內…
notIn:
```

奇偶

```
mov eax, n
test eax, 1
jz  isEven          ; 最低位 0 → 偶數
isEven:
```

是否為 k 的倍數

```
mov eax, n
xor edx, edx
mov ecx, 5
div ecx
cmp edx, 0
je  isMul           ; 餘數 0 → 是倍數
isMul:
```

多路分支（switch）

```
cmp n, 0
je  c0
cmp n, 1
je  c1
  ; …其他…
  jmp sEnd
c0:                 ; …case 0…
  jmp sEnd
c1:                 ; …case 1…
sEnd:
```

三數最大值放 ax

```
mov ax, 7
mov bx, 42
mov cx, 15
cmp ax, bx
jge m1
xchg ax, bx
m1: cmp ax, cx
jge m2
xchg ax, cx
m2:
```

4 亂數

a ~ b 的亂數 → edx

```
invoke time, 0
invoke srand, eax   ; 洗牌：只做一次
invoke rand         ; 結果 → eax
xor edx, edx
mov ecx, 96         ; 個數 = b-a+1（3~98）
div ecx
add edx, 3          ; + a
```

9 作業／小考完整程式

作業4-1 猜數字（重玩、0/-1 結束）

```
cnt dword 1
mxR dword ?
tar dword ?
guess dword ?
askLimit BYTE "max (0/-1 quit): ", 0
askGuess BYTE "guess: ", 0
showCnt  BYTE "correct! %d times", 10, 0
gameOver BYTE "game over", 10, 0
showBig  BYTE "too big", 10, 0
showSmol BYTE "too small", 10, 0
; ---- .code ----
  INVOKE time, 0
  INVOKE srand, eax ; 洗牌一次
while_loop:
  INVOKE printf, ADDR askLimit
  INVOKE scanf, ADDR formatIn, ADDR mxR
  cmp mxR, 0
  je game_over
  cmp mxR, -1
  je game_over
  call rand
  xor edx, edx
  div mxR           ; 餘數 0~mxR-1
  inc edx
  mov tar, edx
  mov cnt, 1        ; 每局重設
guess_loop:
  INVOKE printf, ADDR askGuess
  INVOKE scanf, ADDR formatIn, ADDR guess
  mov eax, guess
  cmp eax, 0
  je game_over
  cmp eax, -1
  je game_over
  cmp eax, tar
  je find_same
  jl find_less
  jg find_greater
find_same:
  INVOKE printf, ADDR showCnt, cnt
  jmp while_loop    ; 重玩
find_less:
  INVOKE printf, ADDR showSmol
  inc cnt
  jmp guess_loop
find_greater:
  INVOKE printf, ADDR showBig
  inc cnt
  jmp guess_loop
game_over:
  INVOKE printf, ADDR gameOver
```

11/2 ArraySum（暫存器傳參數）

```
arrS DWORD 10000h,20000h,3000h
; ---- .code ----
  mov esi, OFFSET arrS
  mov ecx, LENGTHOF arrS
  call ArraySum     ; 結果在 eax
; ---- main ENDP 之後 ----
ArraySum PROC
  push esi
  push ecx
  mov eax, 0
AS1: add eax, [esi] ; esi 是位址，不 ×4
  add esi, TYPE DWORD
  loop AS1
  pop ecx           ; 反順序 pop
  pop esi
  ret
ArraySum ENDP
```

⑤ 迴圈

跑 n 次（loop）

```
mov n, 4
mov ecx, n          ; 只能用 ecx，不能是 0
xor eax, eax
L1:                 ; 開頭：標籤（有冒號）
  add eax, ecx
  loop L1           ; 結尾：ecx-1，≠0 跳回
```

i 從 1 到 n（while）

```
mov ebx, 1          ; i
W1: cmp ebx, n
jg  W1end           ; i > n 結束
  ; …用 ebx 做事…
  inc ebx
  jmp W1
W1end:
```

倒數、每次 -2

```
mov ecx, 7
D1:                 ; …用 ecx…
sub ecx, 2
cmp ecx, 0
jg  D1
```

巢狀迴圈（兩層都用 ecx）

```
mov ecx, 3
O1: push ecx        ; 存外圈
  mov ecx, 2
I1:                 ; …內圈…
  loop I1
  pop ecx           ; 還原外圈
  loop O1
```

⑥ 陣列

宣告

```
arr  dword 5, 3, 9, 1 ; 有初值
arr2 dword 4 DUP(?) ; 4 格空的
buf  byte 100 DUP(0) ; 字串空間
; LENGTHOF 個數  SIZEOF 位元組數
; TYPE 一格大小
```

取 / 存第 i 個（i 在 esi）

```
mov esi, 2
; dword×4 word×2 byte×1
mov eax, arr[esi*4]
mov arr2[esi*4], eax
mov eax, arr[8]     ; 直接寫位元組位移
```

加總（esi 當索引）

```
mov ecx, LENGTHOF arr
mov esi, 0
xor eax, eax
S1: add eax, arr[esi*4]
  inc esi
  loop S1
```

加總（esi 當位址）

```
mov esi, offset arr
mov ecx, LENGTHOF arr
xor eax, eax
S2: add eax, [esi]  ; 不再 ×4
  add esi, 4
  loop S2
```

找最大值

```
mov esi, 0
mov ecx, LENGTHOF arr
mov eax, arr[0]
X1: cmp arr[esi*4], eax
  jle X2            ; 無號用 jbe
  mov eax, arr[esi*4]
X2: inc esi
  loop X1
```

兩陣列相加 arr2[i] = arr[i] + arr[i]

```
mov esi, 0
mov ecx, LENGTHOF arr
P1: mov eax, arr[esi*4]
  add eax, arr[esi*4]
  mov arr2[esi*4], eax
  inc esi
  loop P1
```

向左旋轉一格

```
mov edx, arr[0]     ; 備份第一個
mov esi, 0
mov ecx, LENGTHOF arr - 1
R1: mov eax, arr[esi*4+4]
  mov arr[esi*4], eax
  inc esi
  loop R1
mov arr[esi*4], edx ; 放最後
```

⑦ 字串

逐字走訪到結尾 0

```
mov esi, offset buf
T1: mov al, [esi]
  cmp al, 0
  je  T2
  ; …處理 al，改完 mov [esi], al…
  inc esi
  jmp T1
T2:
```

大小寫轉換（al）

```
or  al, 20h         ; 大→小（add al, 20h）
and al, 0DFh        ; 小→大（sub al, 20h）
xor al, 20h         ; 互換
```

字串長度

```
invoke strlen, addr buf ; 長度 → eax
```

比對兩字串

```
mov esi, offset buf
mov edi, offset buf
mov ecx, 5
cld
repe cmpsb
je  same            ; 全部一樣
same:
```

⑧ 程序（副程式）

STDCALL：定義與呼叫

```
; 寫在 main ENDP 之後
F PROC STDCALL p:DWORD, q:DWORD
  mov eax, p        ; p=[ebp+8] q=[ebp+12]
  add eax, q        ; 回傳值放 eax
  ret               ; 自動 ret 8
F ENDP
; 呼叫：push q / push p / call F
;       （stdcall 不用 add esp）
; 或 invoke F, 3, 4，但要先寫：
; F PROTO STDCALL :DWORD, :DWORD
```

C 慣例：定義與呼叫

```
G PROC C p:DWORD
  mov eax, p
  ret
G ENDP
; 呼叫：push 5 / call G / add esp, 4
```

讀 OF 旗標

```
pushfd
pop eax
and eax, 0800h      ; ≠0 代表 OF=1
```

! 寫完檢查

scanf 給 **addr**、printf 數字給值 · call 會弄壞 eax ecx edx · loop 只用 ecx · mem,mem 不行 · div/mul 不接數字 · 除法前清 edx · 每個分支結尾都 jmp · push/pop 成對、順序相反 · stdcall 不 add esp、C 要 · 最後 ExitProcess

⑨ 作業／小考完整程式（續）

作業4-2 核心：四路分支、記錄錯題、分數

```
op dword ?
numA dword ?
numB dword ?
ans dword ?
ansInput dword ?
cntQ dword 1
correct dword 0
wrong dword 0
wCnt dword 4 DUP(0)
wA dword 50 DUP(0)
fmtAdd BYTE "%d) %d + %d = ", 0
fmtSub BYTE "%d) %d - %d = ", 0
fmtMul BYTE "%d) %d * %d = ", 0
fmtDiv BYTE "%d) %d / %d = ", 0
qFmt dword OFFSET fmtAdd, OFFSET fmtSub
     dword OFFSET fmtMul, OFFSET fmtDiv
; ---- .code ----
; op = rand%4、numA/numB 用 ⑦ 亂數
  cmp op, 0
  je do_add
  cmp op, 1
  je do_sub
  cmp op, 2
  je do_mul
do_div:             ; 先定商，A=B×商
  mov eax, numA
  mov ans, eax      ; 商
  mul numB
  mov numA, eax     ; 一定整除
  jmp ask
do_add:
  mov eax, numA
  add eax, numB
  mov ans, eax
  jmp ask           ; 都跳匯合點
do_sub:
  mov eax, numA
  cmp eax, numB
  jge sub_ok        ; 不可小減大
  mov ebx, numB
  mov numA, ebx
  mov numB, eax
sub_ok:
  mov eax, numA
  sub eax, numB
  mov ans, eax
  jmp ask
do_mul:
  mov eax, numA
  mul numB
  mov ans, eax
ask:
  mov ebx, op       ; 用 op 挑格式
  invoke printf, qFmt[ebx*4],
         cntQ, numA, numB ; 逗號結尾可換行
  invoke scanf, addr fmtD, addr ansInput
  mov eax, ansInput
  cmp eax, ans
  jne q_wrong
  inc correct
  jmp q_next
q_wrong:
  mov ebx, wrong    ; 存到第 wrong 格
  mov eax, numA
  mov wA[ebx*4], eax
  inc wrong
  mov ebx, op
  inc wCnt[ebx*4]   ; 該類錯 +1
q_next:
; 分數 = correct×100÷題數；等第由差往好升級
  mov eax, correct
  mov ecx, 100
  mul ecx
```

作業3 質數判斷

```
n dword ?
msgPrime byte "Prime", 0Ah, 0
msgNotPrime byte "Not Prime", 0Ah, 0
; ---- .code ----
  INVOKE scanf, OFFSET formatIn, OFFSET n
  cmp n, 1
  jle not_prime_label
  cmp n, 2
  je is_prime_label
  mov ebx, 2
check_loop:
  mov eax, n
  mov edx, 0        ; 除法前清 edx
  div ebx
  cmp edx, 0        ; 餘數 0 → 整除
  je not_prime_label
  inc ebx
  cmp ebx, n
  jl check_loop
is_prime_label:
  INVOKE printf, OFFSET msgPrime
  jmp end_program
not_prime_label:
  INVOKE printf, OFFSET msgNotPrime
end_program:
```

作業3 1*1+…+n*n（巢狀、不用乘法）

```
n dword ?
sumSq dword 0
; ---- .code ----
  INVOKE scanf, OFFSET formatIn, OFFSET n
  mov ecx, n
  mov ebx, 1
outer_loop:
  push ecx          ; 存外圈 ecx
  mov ecx, ebx      ; 內圈跑 ebx 次
  mov edx, 0
inner_loop:
  add edx, ebx      ; ebx 加 ebx 次
  loop inner_loop
  add sumSq, edx
  inc ebx
  pop ecx           ; 還原外圈
  loop outer_loop
  INVOKE printf, OFFSET formatOut, sumSq
```

作業2 費氏數列前 11 項

```
fib dword 11 DUP(?)
; ---- .code ----
  mov fib[0], 1
  mov fib[4], 1
  mov ecx, 9        ; 還要算 9 項
  mov esi, 8        ; 從 fib[2] 開始
fib_loop:
  mov eax, fib[esi - 4]
  add eax, fib[esi - 8]
  mov fib[esi], eax ; f[i]=f[i-1]+f[i-2]
  add esi, 4
  loop fib_loop
```